

Introduction to simulations workshop

Paul Whitford

10/12/22

Objective

- We are going to perform some simulations of Cox2
 - Using an all-atom model
 - Using a C-alpha model

Software to be used

- smog2 – package that reads a PDB file and then defines a structure-based “SMOG” model
- Python – we have already installed libraries called OpenMM and OpenSMOG, which will be used to perform the simulation
- vmd – “visual molecular dynamics” allows us to visualize the trajectories

Caution!

- Commands will be shown in the font `courier`
- DO NOT COPY AND PASTE COMMANDS.
Hidden characters will interfere with the commands. Must type them!

Controlling temperature

- Temperature is the parameter that governs the average kinetic energy

$$\left\langle \frac{mv^2}{2} \right\rangle = \frac{3}{2} k_B T$$

- Left side is the average kinetic energy of a single atom.
 k_B is the Boltzmann constant
- T is the temperature
- In a simulation, specifying a temperature is accomplished by ensuring that the average kinetic energy is correct

Let's explore the role of temperature in molecular dynamics

At room temperature, proteins will constantly deform. We would like to inspect those fluctuations.

Simulate Cox2

Steps to get started:

- Start VirtualBox
- Launch the workshop image
- Login (if needed)
- Open a terminal

When in the terminal

Before we start, give the command:

```
PATH=$PATH:/home/student/BIN/smog2.4.5-pre1/bin
```

When in the terminal

Issue the following command to retrieve all workshop files

```
wget http://smog-server.org/workshop_RCC/active.tar
```

“untar” the files with:

```
tar xvf active.tar
```

Now, you should have a directory called “active”

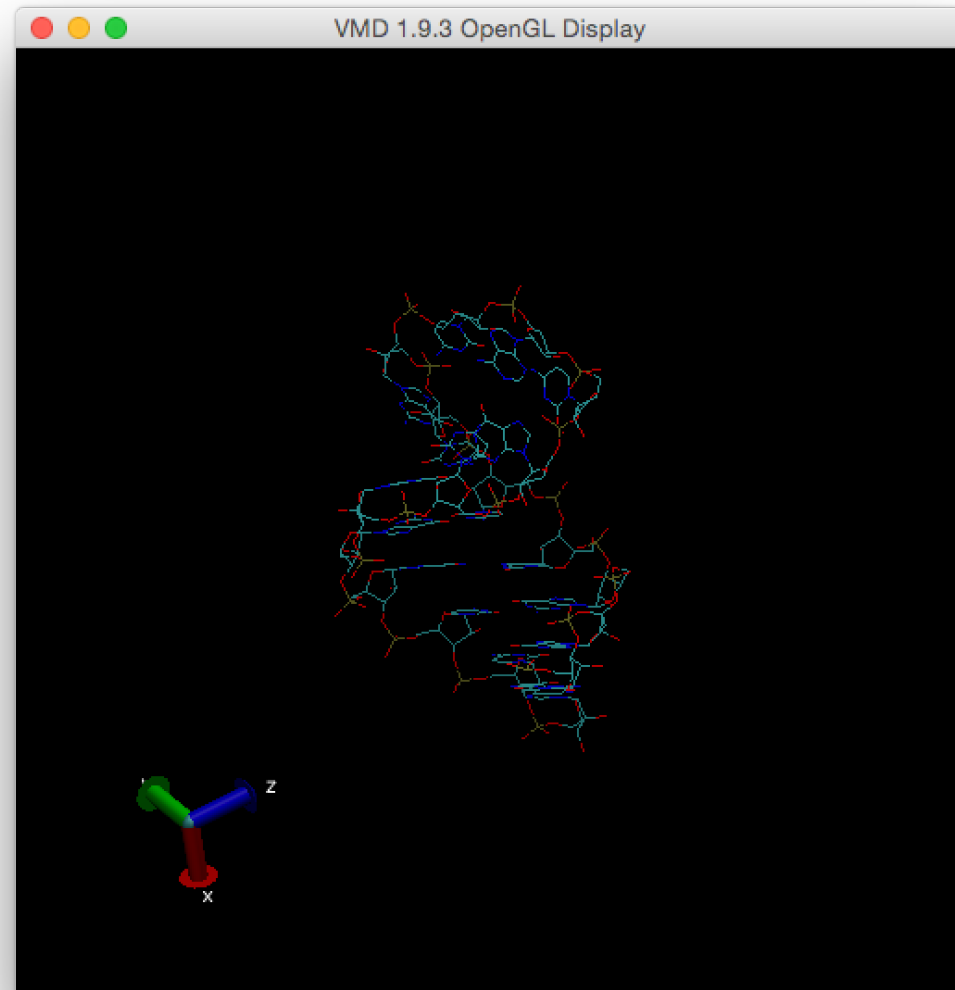
Go into “active” with :

```
cd active
```

Simulate a protein

Visualize the protein

```
vmd <insert PDB name>
```



Prepare a simulation

First, we need to generate a model. For this we will use the “SMOG” software to generate an all-atom structure-based model. However, before we can do that, we need to clean up the PDB file:

```
smog_adjustPDB -i Cox2_aspirin_active_site.pdb \  
-Altloc -removeH -map coxMap -o cox.adjusted.pdb
```

This takes care of peculiar aspects of PDB files that can confuse simulation software.

Next, we need to generate a potential energy function.

```
smog2 -i cox.adjusted.pdb \  
-t cox_model.AA -dname cox.AA -openSMOG
```

“-t” indicates what model we want to use

“-i” indicates that we want to generate a model based on this pdb file

“-dname cox.AA” indicates that all output files should be named cox.AA.<suffix>

“-OpenSMOG” indicates that we want to generate files for a specific simulation tool (OpenSMOG)

If smog is successful, you can use “ls” and see several new files

gro (coordinate) file

Gro file for a structure based model, generated with SMOG Version 2.0.4t

1230

1C	P	1	-0.449	-1.744	-1.435
1C	O1P	2	-0.366	-1.748	-1.558
1C	O2P	3	-0.427	-1.637	-1.335
1C	O5*	4	-0.604	-1.742	-1.478
1C	C5*	5	-0.642	-1.752	-1.616
1C	C4*	6	-0.793	-1.765	-1.635
1C	O4*	7	-0.852	-1.849	-1.537
1C	C3*	8	-0.864	-1.632	-1.619
1C	O3*	9	-0.876	-1.573	-1.749
1C	C2*	10	-1.005	-1.671	-1.571
1C	O2*	11	-1.100	-1.664	-1.678
1C	C1*	12	-0.989	-1.812	-1.522
1C	N1	13	-1.032	-1.821	-1.382
1C	C2	14	-1.167	-1.827	-1.359
1C	O2	15	-1.245	-1.824	-1.454
1C	N3	16	-1.211	-1.838	-1.230
1C	C4	17	-1.123	-1.842	-1.129
1C	N4	18	-1.168	-1.853	-1.004
1C	C5	19	-0.982	-1.835	-1.153
1C	C6	20	-0.942	-1.825	-1.281
2U	P	21	-0.944	-1.428	-1.769
2U	O1P	22	-0.915	-1.382	-1.907
2U	O2P	23	-0.906	-1.343	-1.654
2U	O5*	24	-1.102	-1.460	-1.758
2U	C5*	25	-1.178	-1.494	-1.875
2U	C4*	26	-1.327	-1.471	-1.852

.top (forcefield) file

Indicates which atom types are in this simulation

Name of the complex

Specific atoms in this system

List of native contacts, native distances and energetic weights

Chemical bonds

internal bookkeeping

bond angles

dihedrals

```
[ defaults ]
;nbfunc comb-rule gen-pairs
      1          1 no

[ atomtypes ]
;name mass      charge      ptype c6      c12
CA    1.000      0.000 A      0.000 0.16777E-04

[ moleculetype ]
;name nrexcl
Macromolecule      3

[ atoms ]
;nr  type  resnr residue atom  cgnr charge  mass
  1  CA      1  VAL  CA      1  0.0    1.000
  2  CA      2  LEU  CA      2  0.0    1.000
...
...

[ pairs ]
  1  578 1  0.30873E-02  0.56583E-03
  1  579 1  0.87833E-02  0.19842E-02
...
...

[ bonds ]
;ai  aj  func  r0(nm)  Kb
  1  2  1  0.38091E+00  0.20000E+05
  2  3  1  0.38222E+00  0.20000E+05
  3  4  1  0.38511E+00  0.20000E+05
...
...

[ exclusions ]
;ai  aj
  1  578
  1  579
  1  582
...
...

[ angles ]
;ai  aj  ak  func  th0(deg)  Ka
  1  2  3  1  0.12475E+03  0.40000E+02
  2  3  4  1  0.11047E+03  0.40000E+02
  3  4  5  1  0.10419E+03  0.40000E+02
...
...

[ dihedrals ]
;ai  aj  ak  al  func  phi0(deg)  Kd(kJ/mol)  mult
  1  2  3  4  1  0.44326E+03  0.10000E+01  1
  1  2  3  4  1  0.13298E+04  0.50000E+00  3
  2  3  4  5  1  0.43680E+03  0.10000E+01  1
...
...

[ system ]
;name
Macromolecule

[ molecules ]
Macromolecule 1
```

Simulate the protein

- We will use OpenSMOG/OpenMM, which are python libraries. Since we have already prepared the scripts, we can run the simulation with a simple command:

```
python run.AA.py
```

- Before we do, let look at run.CA.py. Open the file with:

```
gedit run.AA.py
```

- After this, let's actually run the system

Visualize your simulation

- Launch VMD
- Load cox.AA.gro
- Show in tube representation
- Load the trajectory file:
`output_cox.AA/cox.AA_trajectory.dcd`
- What does the simulation look like? Does it unfold?

Congrats!

- You have just simulated a protein!